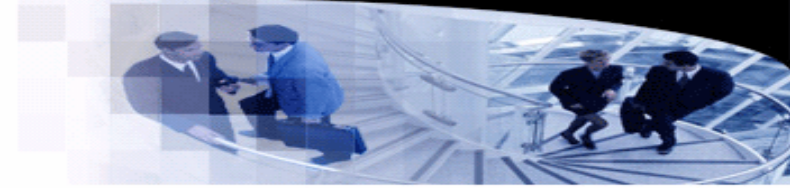




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

Lecture 25. Local Optimizations

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

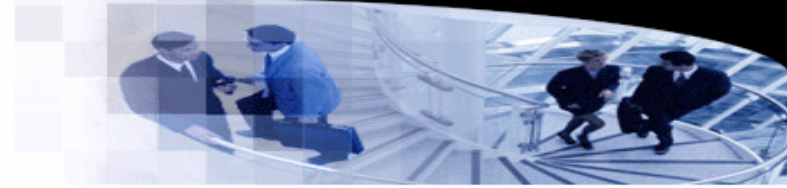
Optimizations

• Introduction

- The passes of compilers that improves the generated code, in terms of performance, resource, energy or security.
- Optimization passes are the most complex part in modern compilers.

```
ymt@DESKTOP-VPDNSDJ:~$ opt-14 -O2 -S -print-pipeline-passes foo.ll
verify,annotation2metadata,forceattrs,inferattrs,function<eager-inv>(lower-expect,simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;no-switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-sink-common-insts>,sroa,early-cse<>,coro-early),openmp-opt,ipscpp,called-value-propagation,globalopt,function(mem2reg),deadargelim,function<eager-inv>(instcombine,simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-sink-common-insts>),require<globals-aa>,function(invalidate<aa>),require<profile-summary>,cgsccl(devirt<4>(inline<only-mandatory>,inline,function-attrs,openmp-opt-cgsccl,function<eager-inv>(sroa,early-cse<memssa>,speculative-execution,jump-threading,correlated-propagation,simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-sink-common-insts>,instcombine,libcall-shrinkwrap,tailcallemim,simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-sink-common-insts>,reassociate,require<opt-remark-emit>,loop-mssa(loop-instsimplify,loop-simplifcfcg,licm,loop-rotate,licm,simple-loop-unswitch<no-nontrivial;trivial>),simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-sink-common-insts>,instcombine,loop(loop-idiom,indvars,loop-deletion,loop-unroll-full),sroa,mldst-motion<no-split-footer-bb>,gvn<>,sccp,bdce,instcombine,jump-threading,correlated-propagation,adce,memcpyopt,dse,loop-mssa(licm),coro-elide,simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;hoist-common-insts;sink-common-insts>,instcombine),coro-split)),globalopt,globaldce,elim-avail-extern,rpo-function-attrs,require<globals-aa>,function<eager-inv>(float2int,lower-constant-intrinsics,loop(loop-rotate,loop-deletion),loop-distribute,inject-tli-mappings,loop-vectorize<no-interleave-forced-only;no-vectorize-forced-only>,loop-load-elim,instcombine,simplifcfcg<bonus-inst-threshold=1;forward-switch-cond;switch-range-to-icmp;switch-to-lookup;no-keep-loops;hoist-common-insts;sink-common-insts>,slp-vectorizer,vector-combine,instcombine,loop-unroll<O2>,transform-warning,instcombine,require<opt-remark-emit>,loop-mssa(licm),alignment-from-assumptions,loop-sink,instsimplify,div-rem-pairs,simplifcfcg<bonus-inst-threshold=1;no-forward-switch-cond;switch-range-to-icmp;no-switch-to-lookup;keep-loops;no-hoist-common-insts;no-sink-common-insts>,coro-cleanup),cgc-profile,globaldce,constmerge,rel-lookup-table-converter,function(annotation-remarks),verify,print
```

Optimizations



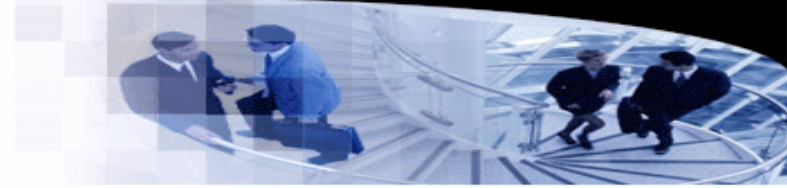
- **Introduction**

- Optimization passes that are performed inside basic blocks are known as **local optimizations**.
- Optimizations that work on the level of CFG are known as **global optimizations**.
- Optimizations of a modern compiler: <https://llvm.org/docs/Passes.html>

- **Local optimizations**

- Constant folding & constant propagation
- Copy propagation
- Dead code elimination
- Common sub-expression elimination
- Strength reduction
- Algebraic simplification

Local Optimizations



- Strength reduction
 - Replace expensive operations with cheaper operations.
- Example:
 - $x * 2 \Rightarrow x \ll 1$
 - $x / 2 \Rightarrow x \gg 1$
 - $x * 17 \Rightarrow (x \ll 4) + x$
 - What about $x * 18$?

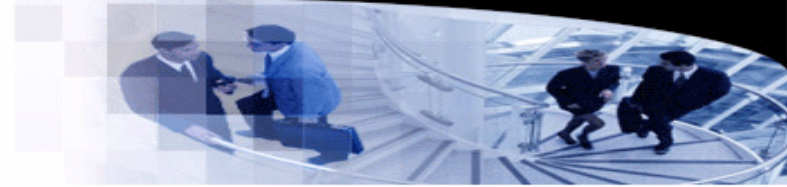
Local Optimizations

- Constant folding

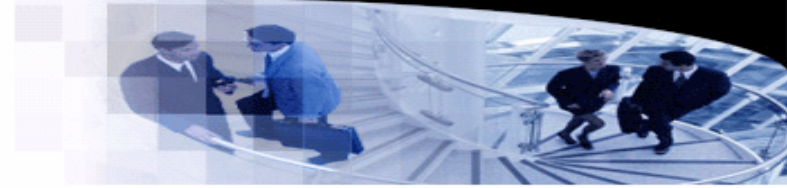
- Compute operations on constants at compile time.

- Example:

- $2 * 2 \Rightarrow 4$
 - $\text{if } (0) \ s \Rightarrow s$
 - $a = 6; \ b = a + 2; \Rightarrow ?$

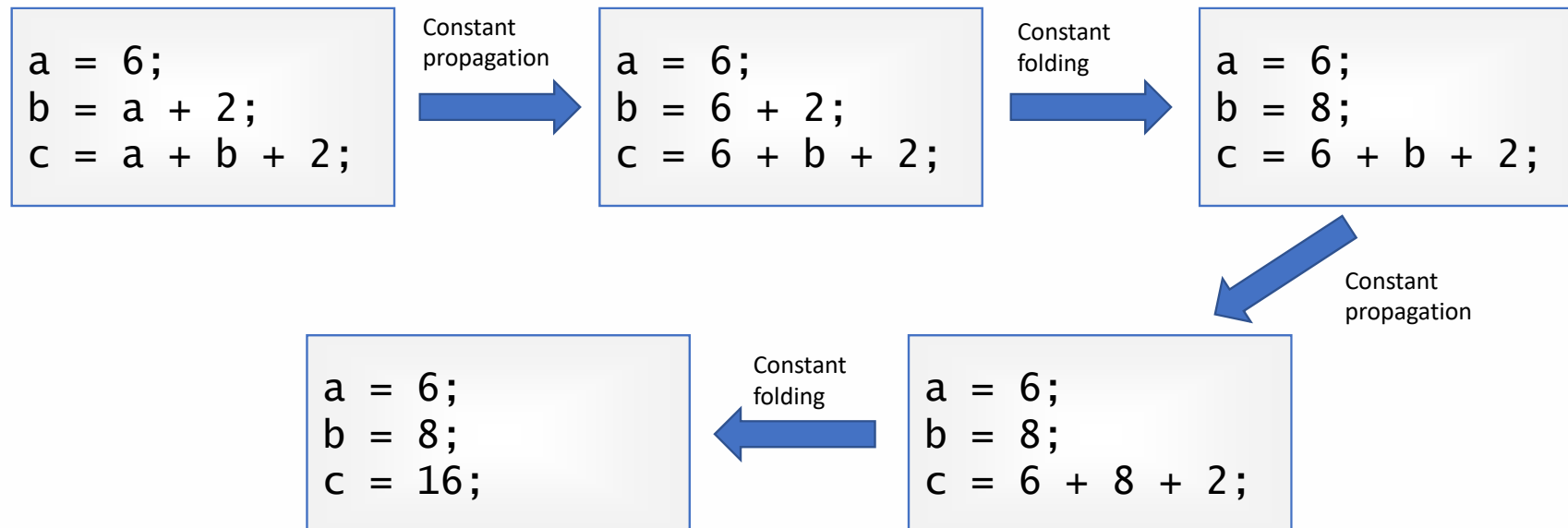


Local Optimizations

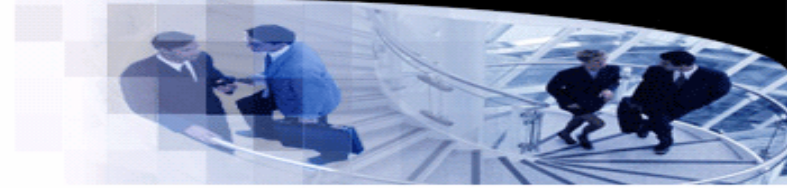


- **Constant propagation**

- If a variable X is defined as a constant C , replace all uses of X with C .
- Combined with constant folding, compilers would try to compute as many constants as possible at compile time.



Local Optimizations



- **Dead code elimination**

- If a piece of code does not affect the result of a program, the code is denoted as a dead code and therefore can be eliminated from the program.
- In imperative programs, a dead code may be a definition of a variable that will never be used.
- In functional programs, a dead code may be an expression that does not appear in any other expressions.

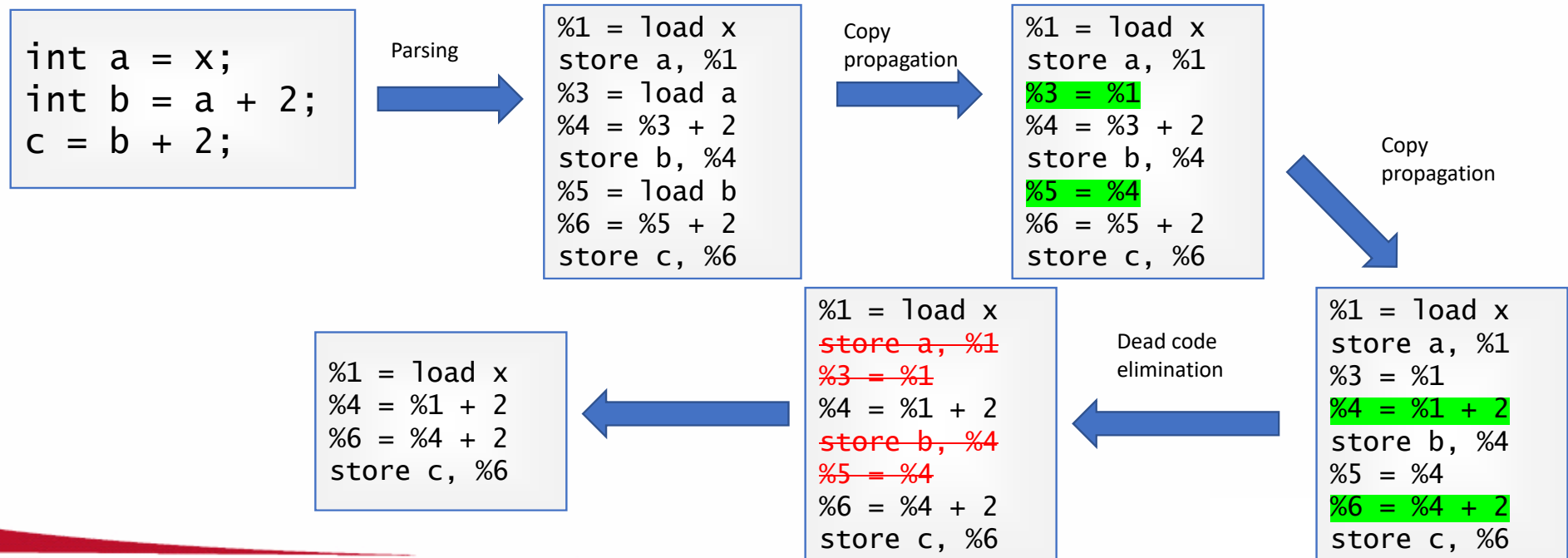
- **Example:**

- `c = 1; c = 2;  $\Rightarrow$  c = 2;`
- `let a = 1 in 2  $\Rightarrow$  2`
- `int c = b + 2; d = c + 2;  $\Rightarrow$  ?` (c will never be used after this assignment.)

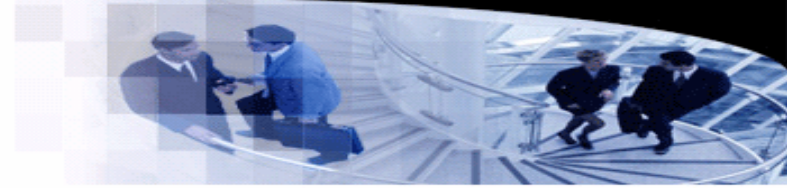
Local Optimizations

- Copy propagation

- When a variable x is defined with another variable (or register), replace the later uses of x with its definition.
- Compilers can find more dead codes after copy propagation.

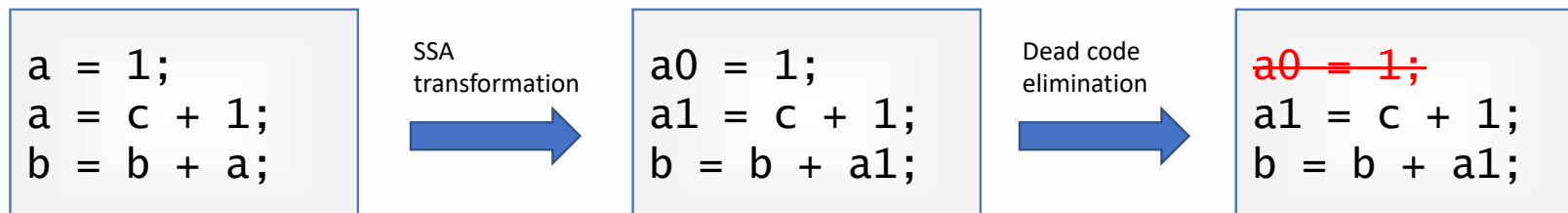


Local Optimizations



- Static single-assignment form (SSA)

- A type of intermediate representation where each variable is defined exactly once.
- However, a variable can be dynamically defined many times, if it is in a loop.
- SSA is used in many modern compilers, including LLVM and GCC.
- SSA provides benefits for compilers to simplify analysis and optimizations.



- How to express the following program in SSA?
 - `if (a == 0) b = 1; else b = 2;`
 - `c = b + 1;`